# Chapter 13 — Transactions and Concurrency

A database is rarely used by one person at a time: a shop's stock table takes hundreds of checkouts per second while reports read it. Two things can go wrong — a failure can stop a change **halfway**, and two users working **at the same time** can trip over each other. This chapter covers the two tools against those problems, **transactions** and **concurrency control**, and shows how PostgreSQL really behaves with live sessions running side by side.

**What you'll learn**

- Transactions, their life cycle, and `BEGIN` / `COMMIT` / `ROLLBACK` / `SAVEPOINT`.
- The four ACID guarantees and how databases implement each.
- Every classic anomaly — dirty read/write, non-repeatable read, phantom, lost update, write skew — **reproduced on a real PostgreSQL server** with two interleaved sessions.
- The four isolation levels in PostgreSQL vs MySQL, SQL Server, Oracle and SQLite.
- Concurrency control: locks and two-phase locking, optimistic validation, timestamp ordering, and MVCC (PostgreSQL's `xmin` / `xmax` from real queries).
- Deadlocks, `SELECT … FOR UPDATE`, `NOWAIT`, and the `SKIP LOCKED` job queue.

**How to run the examples.** All code lives in `code/`. The single-session files run on the book's private PostgreSQL 14 server (port 5544):

```
bash tools_run_sql.sh sql                                   # runs every chapter's .sql files
bash tools_pg.sh psql -d <db> -f sql/chapters/13-transactions-and-concurrency/code/01_commit_and_rollback.sql
```

The two-session demos are in `code/concurrency_demo.py`: connections **A** and **B** send statements in a fixed order; a statement expected to block runs in a background thread while a third *monitor* connection checks `pg_stat_activity` to prove it is waiting on a lock:

```
cd sql/chapters/13-transactions-and-concurrency/code
../../../../.venv-books/bin/python concurrency_demo.py            # all scenarios (~2 s)
../../../../.venv-books/bin/python concurrency_demo.py writeskew  # just one group
```

In the output, `A| …` is a statement from session A, `->` the server's answer, `..` a monitor observation. Transaction IDs differ on every run. Each scenario starts from fresh sample data (`00_schema.sql`):

```
CREATE TABLE accounts (id int PRIMARY KEY, owner text NOT NULL,
                       balance numeric(10,2) NOT NULL CHECK (balance >= 0));
INSERT INTO accounts VALUES (1, 'asha', 500), (2, 'ben', 300), (3, 'chen', 50);

CREATE TABLE products (id int PRIMARY KEY, name text NOT NULL,
                       stock int NOT NULL CHECK (stock >= 0), version int NOT NULL DEFAULT 1);
INSERT INTO products VALUES (1, 'concert ticket', 10, 1), (2, 'poster', 3, 1);

CREATE TABLE on_call (doctor text PRIMARY KEY, shift text NOT NULL, on_duty boolean NOT NULL);
INSERT INTO on_call VALUES ('dr_rao', 'night', true), ('dr_kim', 'night', true), ('dr_lee', 'day', true);

CREATE TABLE jobs (id int PRIMARY KEY, payload text NOT NULL,
                   status text NOT NULL DEFAULT 'queued', claimed_by text);
INSERT INTO jobs (id, payload) SELECT g, 'email #' || g FROM generate_series(1, 6) g;
```

---

## 13.1 What a transaction is

### What it is

A **transaction** is a group of database operations treated as **one logical unit of work**: either every operation takes effect, or none does. It ends with a **commit** (all changes become permanent and visible to others) or an **abort**/rollback (every change is undone, as if it never ran).

*Analogy:* a transaction is like a shopping cart at a self-checkout. You can scan ten items, but nothing is paid for until you press "pay". If the card is declined, the whole cart is cancelled — you never end up paying for seven of the ten items.

## The problem it solves

Sending ₹100 from Asha to Ben needs two writes: subtract 100 from Asha, add 100 to Ben. If the server crashes or a rule is broken **between** them, Asha has lost 100 and Ben never received it. A transaction makes the pair all-or-nothing, which also makes **retrying safe**: after an abort there are no half-applied leftovers, so the application simply runs it again.

### ACID in one table

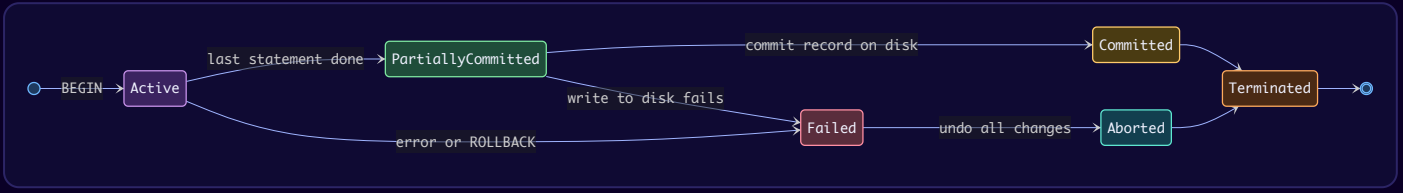
The guarantees a transaction gives are summarised by the acronym **ACID** (coined by Härder and Reuter in 1983). Each letter gets its own section below.

| Letter | Guarantee | Main mechanism |
|---|---|---|
| **A**tomicity | All or nothing — no half-done transfers | Undo information / WAL, shadow copies |
| **C**onsistency | Valid state → valid state (constraints, invariants hold after commit) | Constraints + correct application logic |
| **I**solation | Concurrent transactions don't see each other's in-progress work | Locks, MVCC, SSI |
| **D**urability | Committed means kept, even after a crash or power loss | WAL flushed to disk, replication |

### Life cycle of a transaction

A transaction moves through a small set of **states**:

| State | What is happening |
|---|---|
| **Active** | Statements are running; changes are not yet final. |
| **Partially committed** | Last statement done; results not yet safely on permanent storage. |
| **Committed** | The commit record is durable; changes are permanent. |
| **Failed** | An error (constraint, crash, deadlock, cancel) stops normal progress. |
| **Aborted** | All changes rolled back; the database is as before the transaction. |
| **Terminated** | Finished (by commit or abort); locks and snapshot released. |




After an abort the application either **restarts** the transaction (for temporary problems such as a deadlock) or **gives up** and reports the error (for logical problems such as "insufficient funds").

## How to write one in SQL

| Statement | Effect |
|---|---|
| `BEGIN` (or `START TRANSACTION`) | Start a transaction block. |
| `COMMIT` | Make all changes permanent. |
| `ROLLBACK` | Undo all changes since `BEGIN`. |
| `SAVEPOINT name` | Mark a point inside the transaction. |
| `ROLLBACK TO SAVEPOINT name` | Undo only the work done after that mark; the transaction stays open. |
| `RELEASE SAVEPOINT name` | Forget the mark (keep the work). |

**Autocommit.** If you do *not* write `BEGIN`, PostgreSQL (like psql, JDBC and most drivers by default) wraps **each single statement** in its own transaction and commits it immediately. So a lone `UPDATE` is already atomic; you need `BEGIN … COMMIT` only when several statements must succeed or fail together. (Python's `psycopg` is the odd one out: by default it opens a transaction for you and waits for `conn.commit()`.)

`code/01_commit_and_rollback.sql`:

```sql
-- 1) Money transfer that commits: both rows change together.
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;    -- debit asha
UPDATE accounts SET balance = balance + 100 WHERE id = 2;    -- credit ben
COMMIT;
SELECT id, owner, balance, (SELECT sum(balance) FROM accounts) AS bank_total
FROM accounts ORDER BY id;

-- 2) A transfer we change our mind about: ROLLBACK restores the old state.
BEGIN;
UPDATE accounts SET balance = balance - 200 WHERE id = 1;
SELECT 'inside txn' AS moment, balance FROM accounts WHERE id = 1;
ROLLBACK;
SELECT 'after rollback' AS moment, balance FROM accounts WHERE id = 1;

-- 3) Savepoints: undo only part of a transaction.
BEGIN;
UPDATE accounts SET balance = balance + 10 WHERE id = 3;     -- keep this
SAVEPOINT bonus;
UPDATE accounts SET balance = balance + 999 WHERE id = 3;    -- oops
ROLLBACK TO SAVEPOINT bonus;                                 -- undo just the oops
COMMIT;
SELECT id, owner, balance FROM accounts WHERE id = 3;
```

Output:

```
BEGIN
UPDATE 1
UPDATE 1
COMMIT
 id | owner | balance | bank_total
----+-------+---------+------------
  1 | asha  |  400.00 |     850.00
  2 | ben   |  400.00 |     850.00
  3 | chen  |   50.00 |     850.00
(3 rows)

BEGIN
UPDATE 1
   moment   | balance
------------+---------
 inside txn |  200.00
(1 row)

ROLLBACK
     moment     | balance
----------------+---------
 after rollback |  400.00
(1 row)

BEGIN
UPDATE 1
SAVEPOINT
UPDATE 1
ROLLBACK
COMMIT
 id | owner | balance
----+-------+---------
  3 | chen  |   60.00
(1 row)
```

The bank total stays 850 (money moved, none created or lost); the session saw its own uncommitted 200, but after `ROLLBACK` the balance is 400 again; and only the +10 survived the savepoint.

**Dialect differences**

| | PostgreSQL | MySQL (InnoDB) | SQL Server | SQLite |
|---|---|---|---|---|
| Start | `BEGIN` / `START TRANSACTION` | `START TRANSACTION` / `BEGIN` | `BEGIN TRAN` | `BEGIN [DEFERRED\|IMMEDIATE\|EXCLUSIVE]` |
| DDL inside a transaction | Transactional ( `CREATE TABLE` can be rolled back) | Causes an implicit commit | Mostly transactional | Transactional |
| After an error mid-transaction | Whole transaction is "aborted" until `ROLLBACK` | Only the failing statement is undone; transaction continues | Depends on `XACT_ABORT` | Only the failing statement is undone |

### Weaker guarantees some systems offer

Many NoSQL and caching systems trade multi-statement transactions away for speed or scaling and offer smaller building blocks:

| Guarantee | What it means | Example |
|---|---|---|
| **Single-object atomicity** | A write to *one* record/document is all-or-nothing, but nothing spans two records. | A single MongoDB document update; a Cassandra row write. |
| **Atomic increment** | The server does read-add-write in one indivisible step, so concurrent `+1`s never lose each other. | Redis `INCR`; SQL `UPDATE … SET n = n + 1`. |
| **Compare-and-set (CAS)** | "Write the new value only if the current value is still what I read." If someone changed it, the write fails and you retry. | Redis `WATCH` / `MULTI`, DynamoDB conditional writes, a SQL `UPDATE … WHERE version = 7` (§13.7). |

They protect **one** object only; keeping several records in step (a transfer, an order plus its stock decrement) needs a real transaction – which is why MongoDB (4.0+) and NewSQL systems added multi-document transactions.

### In the interview

> **30-second answer:** "A transaction is a unit of work that fully commits or fully rolls back, with the ACID guarantees. In SQL: `BEGIN … COMMIT/ROLLBACK`, savepoints for partial rollback; without `BEGIN` each statement autocommits."

- *Why not undo step 1 myself on error?* The crash may kill your program too; the database's log handles it even after a power failure.
- *Is a single `UPDATE` atomic?* Yes – in autocommit mode it is its own transaction, and even a multi-row `UPDATE` changes all matching rows or none.

### Self-check

1. Name the six transaction states. → Active, partially committed, committed, failed, aborted, terminated.
2. What does autocommit mean? → Each statement is its own transaction, committed as soon as it succeeds.
3. What does `ROLLBACK TO SAVEPOINT s` keep? → Everything done before `SAVEPOINT s`; the transaction stays open.

---

## 13.2 Atomicity

### What it is

**Atomicity** means a transaction is **indivisible**: all of its changes are applied (commit), or none are (rollback). There is no visible "half-done" state.

### The problem it solves

Take accounts X = 30 and Y = 100 and a transfer of 10 from X to Y. The debit succeeds (X = 20), then the credit fails. Without atomicity the database is left with X = 20, Y = 100: ten units have disappeared. With atomicity the failure rolls back the debit too, leaving X = 30, Y = 100 – exactly as before.

### How it works – three implementations

**1. Shadow copy (the simplest idea, used by early systems and some editors).**

1. When a transaction begins, copy the whole database file; the copy is the **shadow**.
2. Make every change in the shadow only; the original is untouched.
3. To commit, atomically switch a **db-pointer** (a small file naming the current database file) to the shadow, then delete the old file.
4. To abort, delete the shadow; the pointer never moved.

So the only commit point is the pointer switch: if every write succeeds but the crash comes just before the switch, or a failure happens midway, the transaction is aborted and the untouched original remains current.

It is impractical for real databases: copying the **entire** database per transaction is slow and wastes storage, and it allows only **one** transaction at a time, so it can't scale. (Its descendant *shadow paging* – copy only

modified pages — was used in System R and lives on in copy-on-write stores such as LMDB.)

**2. Logging (how PostgreSQL, MySQL, SQL Server and Oracle do it).** Changes go into in-memory pages, and every change is also described in the **write-ahead log (WAL)** (§13.5). If the transaction aborts or the system crashes before commit, the engine undoes the changes from its log — or, in PostgreSQL, simply marks the transaction ID *aborted* so its row versions are ignored (§13.14).

**3. Across machines.** When one transaction touches several databases, **two-phase commit (2PC)** makes them agree: a coordinator asks every participant to *prepare* (make the change durable but not final) and, only if all say yes, tells them to *commit* (chapter 15; PostgreSQL's `PREPARE TRANSACTION`). Optimistic concurrency control (§13.9) also applies a transaction's writes all at once, at commit, only if no conflict was found.

## Code: an error aborts the whole transaction

`code/02_atomicity_on_error.sql` — Chen has 50 and tries to send 80. The credit to Asha runs first and succeeds; then the debit breaks the `CHECK (balance >= 0)` rule:

```
\set ON_ERROR_STOP off
BEGIN;
UPDATE accounts SET balance = balance + 80 WHERE id = 1;   -- credit asha: OK
UPDATE accounts SET balance = balance - 80 WHERE id = 3;   -- debit chen: violates CHECK
SELECT balance FROM accounts WHERE id = 1;                  -- refused: txn is aborted
COMMIT;                                                     -- turns into ROLLBACK
SELECT id, owner, balance FROM accounts ORDER BY id;        -- nothing changed

BEGIN;
UPDATE accounts SET balance = balance + 5 WHERE id = 2;
SAVEPOINT before_risky;
INSERT INTO accounts VALUES (1, 'dup', 1);                  -- duplicate primary key
ROLLBACK TO SAVEPOINT before_risky;
COMMIT;
SELECT id, owner, balance FROM accounts WHERE id = 2;
```

Output (psql file path shortened):

```
BEGIN
UPDATE 1
psql:02_atomicity_on_error.sql:10: ERROR:  new row for relation "accounts" violates check constraint
"accounts_balance_check"
psql:02_atomicity_on_error.sql:11: ERROR:  current transaction is aborted, commands ignored until end of
transaction block
ROLLBACK
 id | owner | balance
----+-------+---------
  1 | asha  |  500.00
  2 | ben   |  300.00
  3 | chen  |   50.00
(3 rows)

BEGIN
UPDATE 1
SAVEPOINT
psql:02_atomicity_on_error.sql:19: ERROR:  duplicate key value violates unique constraint "accounts_pkey"
ROLLBACK
COMMIT
 id | owner | balance
----+-------+---------
  2 | ben   |  305.00
(1 row)
```

After an error PostgreSQL marks the transaction **aborted** and refuses further commands; `COMMIT` is answered with `ROLLBACK`, undoing Asha's successful +80 too. Rolling back to a savepoint clears the error and keeps Ben's +5.

### Common mistakes

- Running the two halves of a transfer as two autocommitted statements.
- Catching the error in application code and carrying on to `COMMIT` – PostgreSQL silently rolls back; MySQL may commit the half that worked.
- Assuming rollback undoes an email sent or card charged inside the transaction – it undoes only **database** changes; record the action in an *outbox* table and send after commit.

### In the interview

> **30-second answer:** "All-or-nothing. Shadow copy (work on a copy, flip a pointer at commit) explains the idea but copies too much and serialises everything; real engines log changes and roll back from the log; across databases you need two-phase commit."

- *Does atomicity say anything about concurrency?* No – ACID's "atomic" is about failures; other users are isolation's job (unlike "atomic" in multithreaded code).

### Self-check

1. In shadow copy, what happens if the crash occurs after all writes but before the pointer update? → The transaction is aborted; the original database is still current.
2. Why doesn't shadow copy scale? → It copies the whole database per transaction and allows one writer at a time.
3. What does PostgreSQL answer to `COMMIT` in an aborted transaction? → `ROLLBACK`.

---

## 13.3 Consistency

### What it is

**Consistency** (the C in ACID) means every committed transaction leaves the database **valid if it started valid**. "Valid" is defined by rules: primary and unique keys, foreign keys, `CHECK` and `NOT NULL` constraints, and business **invariants** such as "money is neither created nor destroyed by a transfer". A transaction that would break a rule must fail and leave the previous valid state untouched; rules may be briefly false only *inside* a transaction.

#### Worked example – a transfer keeps the total

Asha has 300, Ben has 400: the total is **700**. Transfer 100 from Asha to Ben:

| Moment | Step | Asha | Ben | Total | Valid? |
|---|---|---|---|---|---|
| t1 | initial state | 300 | 400 | 700 | yes |
| t2 | `BEGIN` | 300 | 400 | 700 | yes |
| t3 | debit Asha 100 | 200 | 400 | **600** | no (intermediate) |
| t4 | credit Ben 100 | 200 | 500 | 700 | yes |
| t5 | `COMMIT` | 200 | 500 | 700 | yes |
| t6 | final state | 200 | 500 | 700 | yes |

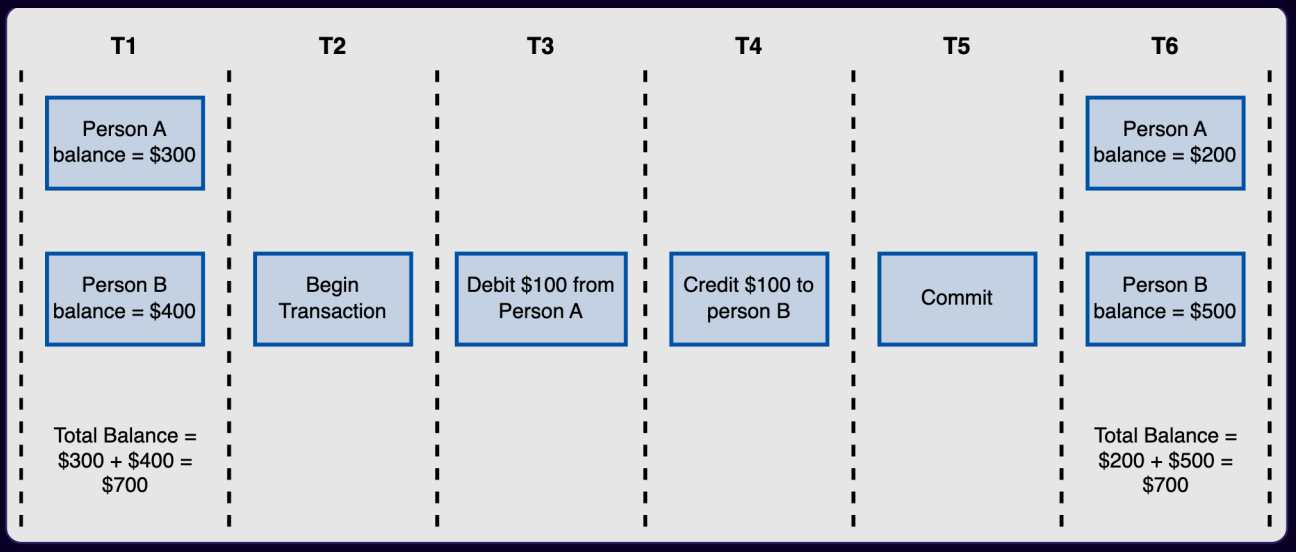



*Figure – the total balance is correct before and after the transaction; only the invisible middle is inconsistent (course diagram).*

The temporary 600 at t3 is harmless: nobody else can see it (isolation) and it can never be the final state (atomicity).

### How it is achieved

1. **Declarative constraints** checked by the database: `PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY`, `CHECK (balance >= 0)`, `NOT NULL` (chapter 9). §13.2's demo showed the `CHECK` rejecting an overdraft.
2. **Pre-transaction checks** in the application or in SQL: e.g., "does Asha have enough money?" before debiting (best done *in the same statement*: `UPDATE … SET balance = balance - 100 WHERE id = 1 AND balance >= 100`).
3. **Post-transaction verification**: deferred constraints (`DEFERRABLE INITIALLY DEFERRED` checks a foreign key only at commit) or audits confirm the invariant, e.g., the ledger still sums to the same total.
4. **Atomicity** and **isolation** stop failures and concurrent transactions from leaving a broken state.

### When consistency can be violated

| Cause | Example | Who stops it |
|---|---|---|
| Constraint violation | Inserting a duplicate primary key | The database rejects the statement |
| Crash mid-transaction | Crash after debit, before credit | Atomicity + recovery roll it back |
| Logic bug | Code transfers more than the balance because it forgot the check | Only a constraint (`CHECK`) or correct code |
| Concurrency without enough isolation | Two withdrawals both see enough money and both proceed | Isolation level / locking (§13.4–13.8) |

## Trade-offs and a note on the word "consistency"

The database enforces only rules it knows about. Cross-row invariants ("at least one doctor on call") can't be a simple `CHECK`; the application enforces them and needs strong enough isolation (§13.8). Hence Kleppmann (DDIA) calls C a property of the *application*, with A, I and D as the database's tools for it.

Also, **ACID consistency is not CAP consistency.** In CAP, "consistency" means *linearizability* – every read sees the latest write across replicas (chapter 15). Same word, different idea.

## In the interview

> **30-second answer:** "Every committed state obeys the rules – keys, checks, business invariants. The database enforces declared constraints, the application the rest; atomicity and isolation keep failures and concurrency from breaking them."

- *Give an invariant a CHECK can't express.* "At least one doctor is on call per shift" – it spans rows; needs SERIALIZABLE or explicit locking.

### Self-check

1. Is a temporarily wrong total inside a transaction a violation? → No; only committed states must be valid.
2. Name three enforcement mechanisms. → Constraints, pre-checks in the statement/app, atomic rollback (plus isolation).
3. Why can concurrency break consistency? → Two transactions each valid alone can together break an invariant if isolation is too weak.

---

## 13.4 Isolation – anomalies and isolation levels

### What it is

**Isolation** controls what one running transaction can see of another's in-progress work. Ideally each behaves **as if it were alone**, but running transactions truly one at a time is too slow, so databases let them overlap and offer **isolation levels**: weaker levels allow more overlap but permit certain **anomalies** (wrong results caused by interleaving); stronger levels forbid more of them at the cost of blocking or aborts.

*Analogy:* cooks sharing one kitchen. "One cook at a time" is full isolation; real kitchens let everyone work with rules (don't take from someone's half-finished dish). The isolation level is how strict the rules are.

#### The anomalies

Timelines below: **T1** and **T2** are concurrent; the account starts at 500.

**1. Dirty read** – reading data another transaction has written but **not committed**.

| Time | T1 | T2 |
|---|---|---|
| 1 | `BEGIN`; balance = 500 – 100 → **400** (uncommitted) | |
| 2 | | reads balance → **400** |
| 3 | `ROLLBACK` → balance is 500 again | |
| 4 | | has acted on 400, a value that never officially existed |

**2. Dirty write** – overwriting a value another transaction has written but **not committed**. T1 adds 100 (500 → 600, uncommitted); T2 adds 100 on top (→ 700); then T1 aborts. Restoring "500" erases T2's deposit; keeping 700 keeps T1's cancelled deposit; the correct 600 can't be reconstructed. Practically every database, at every level, prevents dirty writes by making the second writer **wait** for the first to finish.

**3. Non-repeatable read (also called read skew)** – the same row read twice in one transaction gives **different committed values**, because another transaction committed a change in between.

| Time | T1 | T2 |
|---|---|---|
| 1 | reads balance → 500 | |
| 2 | | balance = 400; `COMMIT` |
| 3 | reads balance again → **400** | |

"Read skew" is the broader name for seeing *different moments in time* in one transaction – e.g., reading account 1 before a transfer and account 2 after it, so the total looks wrong. §13.6 shows exactly this on PostgreSQL.

**4. Phantom read** – the same **query** run twice returns a **different set of rows**, because another transaction inserted or deleted rows that match the condition.

| Time | T1 | T2 |
|---|---|---|
| 1 | `SELECT count(*) … WHERE balance >= 100` → 2 | |
| 2 | | `INSERT` a matching row; `COMMIT` |
| 3 | same query → **3** (a "phantom" row appeared) | |

**5. Lost update** – two transactions each **read** a value, **compute** a new value, and **write** it; the second write silently overwrites the first.

| Time | T1 | T2 |
|---|---|---|
| 1 | reads stock → 10 | |
| 2 | | reads stock → 10 |
| 3 | writes 10 − 1 = 9 | |
| 4 | | writes 10 − 1 = 9 |
| | *Two items sold, stock only fell by one.* | |

**6. Write skew** – two transactions read the **same rows**, each decides based on what it read, and each writes **a different row**. Neither overwrote the other, yet together they break a rule each checked: two on-call doctors each see "2 on call", both go off duty, nobody is left (§13.8). A lost update is the special case where both write the *same* row.

### The isolation levels

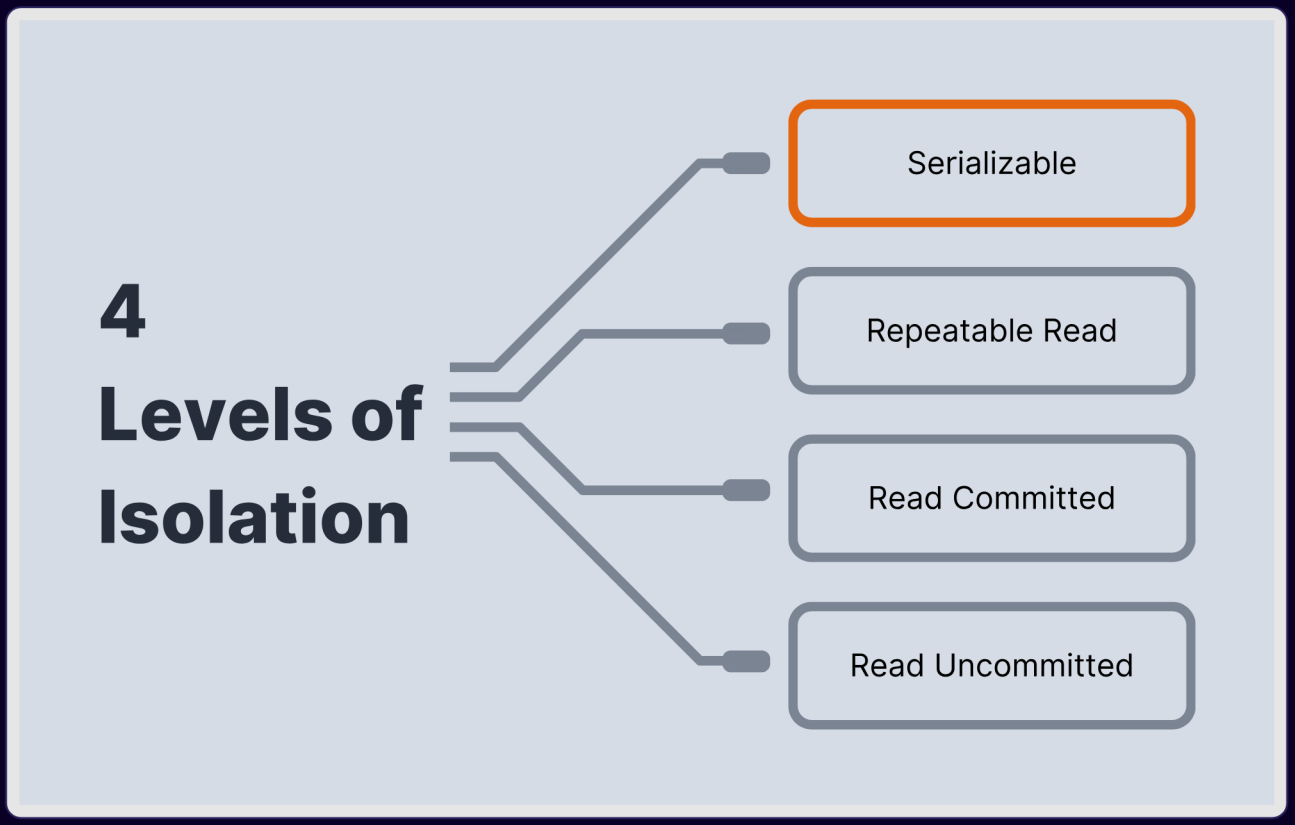



*Figure – the four SQL-standard isolation levels (course diagram).*

The SQL standard defines four levels by which of the first three read anomalies they must prevent:

| Level | Dirty read | Non-repeatable read | Phantom |
|---|---|---|---|
| Read Uncommitted | possible | possible | possible |
| Read Committed | prevented | possible | possible |
| Repeatable Read | prevented | prevented | possible |
| Serializable | prevented | prevented | prevented |

That is the *minimum*; real engines often give more, and the standard ignores lost updates and write skew. What **PostgreSQL** actually provides (verified in §13.6–13.8):

| Level in PostgreSQL | Dirty read | Dirty write | Non-repeatable / read skew | Phantom | Lost update | Write skew | How it's done |
|---|---|---|---|---|---|---|---|
| Read Uncommitted | **no** (treated as Read Committed) | no | yes | yes | yes | yes | new snapshot per statement |
| **Read Committed** (default) | no | no | yes | yes | yes (read-then-write in app) | yes | new snapshot per statement + row locks for writes |
| Repeatable Read | no | no | no | **no** (stronger than the standard) | no – aborts with a serialization error | **yes** | one snapshot per transaction (snapshot isolation) |
| Serializable | no | no | no | no | no | no – aborts one | snapshot isolation + SSI conflict tracking |

Two mechanisms matter. At **Read Committed**, MVCC engines hand readers the last committed version so they never wait, while lock-based engines (SQL Server unless `READ_COMMITTED_SNAPSHOT` is on) make readers wait for the writer. **Serializable** (outcome equals *some* one-at-a-time order) is built with **two-phase locking** (MySQL, SQL Server) or **Serializable Snapshot Isolation (SSI)**, which tracks read/write dependencies and aborts a transaction when needed (PostgreSQL 9.1+) – so always be ready to retry. Read Uncommitted (SQL Server's `NOLOCK` hint) is almost never used.

### Other engines at a glance

| Engine | Default level | Notes |
|---|---|---|
| PostgreSQL | Read Committed | RR = snapshot isolation (no phantoms); Serializable = SSI. |
| MySQL InnoDB | Repeatable Read | Plain `SELECT` reads a snapshot; locking reads (`FOR UPDATE`, `UPDATE`) read the *latest* row with next-key locks against phantoms. Does **not** abort lost updates at RR – use `FOR UPDATE` or atomic updates. |
| SQL Server | Read Committed (locking) | Optional `READ_COMMITTED_SNAPSHOT` and `SNAPSHOT` isolation use row versions in tempdb. |
| Oracle | Read Committed | Its "SERIALIZABLE" is actually snapshot isolation (write skew possible). |
| SQLite | Serializable | Only one writer at a time for the whole database file; WAL mode lets readers run during a write. |

## How to choose and set a level

```
BEGIN ISOLATION LEVEL REPEATABLE READ;                -- for one transaction
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;         -- first statement inside BEGIN
SET default_transaction_isolation = 'serializable';   -- for the session
SHOW transaction_isolation;
```

| If you need… | Use |
| --- | --- |
| Normal OLTP, single-statement updates ( `SET x = x + 1` ) | Read Committed |
| A consistent report or export across many queries | Repeatable Read (often `READ ONLY` ) |
| Read-then-write logic on the same row | `SELECT … FOR UPDATE` at Read Committed, or Repeatable Read + retry |
| Invariants that span rows (no double booking, at least one on call) | Serializable + retry, or explicit locks |

### In the interview

> **30-second answer:** "Isolation levels trade correctness for concurrency; the standard defines four by which of dirty reads, non-repeatable reads and phantoms they allow. Postgres never allows dirty reads, its Repeatable Read is snapshot isolation (no phantoms or lost updates, but write skew), and Serializable uses SSI to abort one transaction of a write skew."

- *What's the difference between a non-repeatable read and a phantom?* Non-repeatable: the *same row* changed value. Phantom: the *set of rows* matching a condition changed (insert/delete).
- *Why isn't everyone on Serializable?* More aborts (or blocking with 2PL) and the application must retry; most workloads are fine at Read Committed with a few targeted locks.

### Self-check

1. Which anomaly involves an insert? → Phantom read.
2. What is PostgreSQL's default level? MySQL's? → Read Committed; Repeatable Read.
3. Two transactions update *different* rows and break a rule together — which anomaly? → Write skew.

---

## 13.5 Durability

### What it is

**Durability** promises that once the database says "committed", the change **stays** — even if the power fails or the process is killed a millisecond later. People act on "committed" (ship the order, release the ticket), so recovery must restore every committed transaction and nothing uncommitted.

*Analogy:* a shop that writes every sale in an ink ledger *before* handing over the receipt; if the till loses power, the ledger still says what was sold.

### How it works

1. **Write-ahead logging (WAL).** A record describing each change must reach the log before the changed data page may be written. At `COMMIT` the database appends a commit record and **flushes** ( `fsync` ) the log to disk; only then does it answer "COMMIT". Data pages are written later, lazily; after a crash, recovery **replays** the log, redoing committed changes that never reached the data files. (One sequential append is much faster than updating scattered pages.)
2. **Checkpoints.** Periodically all dirty pages are written to the data files and "everything up to log position X is on disk" is recorded, so recovery replays only the log *after* the latest checkpoint (PostgreSQL: `checkpoint_timeout` , default 5 min, and `max_wal_size` ).
3. **Synchronous disk writes.** A commit is acknowledged only after the log is on stable storage, not just in RAM or the OS cache. PostgreSQL's `synchronous_commit = on` and `fsync = on` (both defaults) guarantee this; `full_page_writes` protects against *torn pages* (a page only partly written when power fails).
4. **Replication.** Disks die, so the WAL is shipped to other machines (PostgreSQL streaming replication); with *synchronous* replication a commit also waits for a replica (chapter 15).

### Code: watching the WAL grow

`code/05_durability_wal.sql` inserts 1,000 rows and measures how far the WAL position (**LSN**, log sequence number) moved:

```
SELECT name, setting FROM pg_settings
WHERE name IN ('fsync', 'synchronous_commit', 'wal_level', 'full_page_writes', 'checkpoint_timeout')
ORDER BY name;

CREATE TEMP TABLE lsn_marks (label text, lsn pg_lsn);
INSERT INTO lsn_marks VALUES ('before', pg_current_wal_lsn());
INSERT INTO accounts SELECT g, 'user' || g, 100 FROM generate_series(10, 1009) g;   -- 1,000 rows
INSERT INTO lsn_marks VALUES ('after', pg_current_wal_lsn());

SELECT pg_size_pretty(pg_wal_lsn_diff(
         (SELECT lsn FROM lsn_marks WHERE label = 'after'),
         (SELECT lsn FROM lsn_marks WHERE label = 'before'))) AS wal_written_for_1000_rows;

BEGIN;
SET LOCAL synchronous_commit = off;     -- faster commit; a crash may lose this txn, never corrupts
UPDATE accounts SET balance = balance + 1 WHERE id = 10;
COMMIT;
```

Output:

```
        name        | setting
--------------------+---------
 checkpoint_timeout | 300
 fsync              | on
 full_page_writes   | on
 synchronous_commit | on
 wal_level          | replica
(5 rows)
...
 wal_written_for_1000_rows
---------------------------
 135 kB
(1 row)
...
```

The 1,000 rows produced about 135 kB of WAL (the exact size can vary slightly by version and settings). The `synchronous_commit = off` transaction returns before its WAL is flushed: a crash within about three times `wal_writer_delay` could lose that one commit, but never corrupt the database — fine for click logging, never for payments. Conversely, a normal committed transfer or order survives a power cut a moment later, because WAL replay restores it.

## Common mistakes

- Turning off `fsync` for speed in production — a crash can then **corrupt** the database, not just lose recent commits.
- Assuming a replica makes a commit durable when replication is asynchronous — the primary can die before shipping it.
- Disks or cloud volumes that lie about flushes (write cache without battery backup).

## In the interview

> **30-second answer:** "Committed data survives crashes: the WAL record is fsynced before the commit is acknowledged, data pages are written later, and restart replays the log from the last checkpoint. Replication protects against losing the machine."

- *Why not write the data pages at commit?* Scattered page writes cost far more than a single sequential append to the log, and the log record alone is enough to redo the change.

### Self-check

1. What must be on disk before `COMMIT` returns? → The transaction's WAL records including the commit record.
2. What does a checkpoint shorten? → Crash recovery (replay starts at the last checkpoint).
3. Does asynchronous replication strengthen durability of the latest commit? → No; only synchronous replication does.

---

## 13.6 Watching isolation anomalies on a real server

This section runs the anomalies from §13.4 on PostgreSQL 14 with two live sessions (A and B) using `code/concurrency_demo.py`. The core of the driver:

```
class Session:
    def __init__(self, name):
        self.name = name
        self.conn = psycopg.connect(f"{BASE} dbname={DB}", autocommit=True)  # we send BEGIN/COMMIT ourselves
        self.pid = self.conn.info.backend_pid

    def run(self, sql):                       # send one statement, print the answer
        print(f"  {self.name}| {sql}  -> {self._exec(sql)}")

    def start(self, sql):                     # a statement we expect to BLOCK: run it in a thread
        print(f"  {self.name}| {sql}")
        box = {}
        t = threading.Thread(target=lambda: box.setdefault("r", self._exec(sql)))
        t.start()
        return self, t, box

class Monitor:                                # a third connection that watches the other two
    def wait_until_blocked(self, s, timeout=5.0):
        # poll pg_stat_activity until session s shows wait_event_type = 'Lock'
        ...
        # SELECT wait_event_type, wait_event, pg_blocking_pids(pid)
        # FROM pg_stat_activity WHERE pid = %s
```

`_exec` returns the rows, the command tag (`UPDATE 1`), or `ERROR <sqlstate>: <message>`. In deadlock messages, it replaces the process IDs with `<A>` / `<B>` so the output is readable.

### Dirty read and dirty write — impossible in PostgreSQL

Session A changes Asha's balance to 0 but doesn't commit. Session B even asks for `READ UNCOMMITTED`:

```
=== Dirty read and dirty write: impossible in PostgreSQL ===
  A| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = 0 WHERE id = 1  -> UPDATE 1
  B| BEGIN ISOLATION LEVEL READ UNCOMMITTED  -> BEGIN
  B| SHOW transaction_isolation  -> read uncommitted
  B| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  B| UPDATE accounts SET balance = balance + 1 WHERE id = 1
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| ROLLBACK  -> ROLLBACK
     -> (B resumes) UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT balance FROM accounts WHERE id = 1  -> 501.00
```

- **No dirty read:** B really is at `read uncommitted`, yet reads the committed **500**, not A's 0 — PostgreSQL implements Read Uncommitted as Read Committed. The read also didn't block: MVCC hands B the older committed version.
- **No dirty write:** B's `UPDATE` of the same row **waits** (wait event `transactionid`: B waits for A's transaction to end). After A's rollback B updates the committed 500 → 501.

### Non-repeatable read / read skew: Read Committed vs Repeatable Read

A reads Asha's balance. Meanwhile B transfers 100 from Asha (account 1) to Ben (account 2) and commits. A then reads both accounts.

```
=== Non-repeatable read / read skew: READ COMMITTED ===
  A| BEGIN ISOLATION LEVEL READ COMMITTED  -> BEGIN
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  B| BEGIN  -> BEGIN
  B| UPDATE accounts SET balance = balance - 100 WHERE id = 1  -> UPDATE 1
  B| UPDATE accounts SET balance = balance + 100 WHERE id = 2  -> UPDATE 1
  B| COMMIT  -> COMMIT
  A| SELECT balance FROM accounts WHERE id = 1  -> 400.00
  A| SELECT balance FROM accounts WHERE id = 2  -> 400.00
  A| COMMIT  -> COMMIT
```

At Read Committed the same row gave **500 then 400** — a non-repeatable read. An auditor adding its first read of account 1 (500) to its read of account 2 (400) would report **900**, though the true total was 800 at every instant: **read skew**.

```
=== Non-repeatable read / read skew: REPEATABLE READ ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  ... (B runs the same transfer as above and commits)
  A| SELECT balance FROM accounts WHERE id = 1  -> 500.00
  A| SELECT balance FROM accounts WHERE id = 2  -> 300.00
  A| COMMIT  -> COMMIT
```

At Repeatable Read A keeps seeing the world as of its **first statement** (not `BEGIN`): 500 and 300, total 800 — consistent, just slightly old.

### Phantom read: Read Committed vs Repeatable Read

```
=== Phantom read: READ COMMITTED ===
  A| BEGIN ISOLATION LEVEL READ COMMITTED  -> BEGIN
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  B| INSERT INTO accounts VALUES (4, 'dina', 900)  -> INSERT 0 1
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 3
  A| COMMIT  -> COMMIT

=== Phantom read: REPEATABLE READ (PostgreSQL prevents it) ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  B| INSERT INTO accounts VALUES (4, 'dina', 900)  -> INSERT 0 1
  A| SELECT count(*) FROM accounts WHERE balance >= 100  -> 2
  A| COMMIT  -> COMMIT
```

At Read Committed a phantom appears (2 → 3). At Repeatable Read the count stays 2 — the standard would *allow* a phantom, but PostgreSQL's snapshot simply doesn't contain Dina's row.

## Common mistakes

- Believing Repeatable Read shows "the latest data" — a long Repeatable Read transaction sees an increasingly stale world (and holds back cleanup, §13.14).
- Running a multi-query report at Read Committed and getting totals that never existed. Use `BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY`.

---

# 13.7 Lost updates and how to prevent them

## The problem

A **lost update** comes from the *read–modify–write* pattern: the application reads a value, computes a new one in its own code, and writes it back; if two copies overlap, the later write wipes out the earlier one. It is the most common concurrency bug in web apps (stock counters, likes, wallet balances, wiki edits).

### Seeing it happen

Two customers buy a concert ticket at the same moment. Each app instance reads the stock, subtracts one in Python/Java, and writes the result:

```
=== Lost update 1: read-modify-write in the application (READ COMMITTED) ===
  A| BEGIN  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1  -> 10
  B| BEGIN  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1  -> 10
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 9
```

Two tickets sold, stock fell only to **9**, and nothing errored. The fix must make read and write one protected step; there are four standard ways.

### Fix 1 – let the database do the arithmetic (atomic update)

```
=== Lost update fix 1: atomic UPDATE ... SET stock = stock - 1 ===
  A| BEGIN  -> BEGIN
  A| UPDATE products SET stock = stock - 1 WHERE id = 1 RETURNING stock  -> 9
  B| BEGIN  -> BEGIN
  B| UPDATE products SET stock = stock - 1 WHERE id = 1 RETURNING stock
     .. B is WAITING (wait_event=transactionid, blocked by A)
     .. pg_locks for B: (transactionid, ExclusiveLock, True), (tuple, ExclusiveLock, True), (transactionid,
ShareLock, False)
  A| COMMIT  -> COMMIT
     -> (B resumes) 8
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's `UPDATE` waits for A's row lock; when A commits, PostgreSQL (at Read Committed) **re-reads the newest row version** and applies `stock - 1` to it: 9 → 8. The `pg_locks` line shows the mechanism: B holds its own transaction-ID lock and a *tuple* lock (first in line for the row) and waits (`granted = False`) for a `ShareLock` on **A's transaction ID** – "wake me when A ends". The row lock itself is recorded in the row (its `xmax`), not in a lock table, so locking a million rows needs no million lock entries.

*Use when* the new value can be computed in SQL (`n = n + 1`, `balance = balance - 100 WHERE balance >= 100`) – the simplest, fastest fix.

### Fix 2 – pessimistic lock: `SELECT … FOR UPDATE`

When the new value needs application logic (call a pricing service, validate a complex rule), lock the row while reading it:

```
=== Lost update fix 2: pessimistic SELECT ... FOR UPDATE ===
  A| BEGIN  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1 FOR UPDATE  -> 10
  B| BEGIN  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1 FOR UPDATE
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
     -> (B resumes) 9
  B| UPDATE products SET stock = 8 WHERE id = 1  -> UPDATE 1
  B| COMMIT  -> COMMIT
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's read waits until A commits and then returns **9**, not 10, so B computes 8. The lock lasts until the end of the transaction — keep such transactions short and never wait for a human while holding one.

### Fix 3 — Repeatable Read detects it (optimistic, by the database)

```
=== Lost update fix 3: REPEATABLE READ detects the conflict ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT stock FROM products WHERE id = 1  -> 10
  B| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  B| SELECT stock FROM products WHERE id = 1  -> 10
  A| UPDATE products SET stock = 9 WHERE id = 1  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| UPDATE products SET stock = 9 WHERE id = 1  -> ERROR 40001: could not serialize access due to concurrent
update
  B| ROLLBACK  -> ROLLBACK
  (B retries the whole transaction)
  ... (B re-reads 9, writes 8, commits)
  M| SELECT stock FROM products WHERE id = 1  -> 8
```

B's snapshot says 10, but the row was changed by a transaction that committed after that snapshot, so PostgreSQL refuses with SQLSTATE **40001** (`serialization_failure`). The application **retries the whole transaction** and writes 8. (MySQL/InnoDB at Repeatable Read raises no such error — its `UPDATE` reads the latest row version — so there use Fix 1 or 2.)

### Fix 4 — optimistic locking with a version column (compare-and-set)

**Compare-and-set (CAS)** means "change the value only if it is still what I read". It holds no lock, so it suits long gaps between read and write (a user editing a form for minutes, several HTTP requests). Add a `version` column and include it in the `WHERE`:

```
=== Lost update fix 4: optimistic locking with a version column (compare-and-set) ===
  A| SELECT stock, version FROM products WHERE id = 1  -> (10, 1)
  B| SELECT stock, version FROM products WHERE id = 1  -> (10, 1)
  A| UPDATE products SET stock = 9, version = version + 1 WHERE id = 1 AND version = 1  -> UPDATE 1
  B| UPDATE products SET stock = 9, version = version + 1 WHERE id = 1 AND version = 1  -> UPDATE 0
  (B sees UPDATE 0 -> someone changed the row; re-read and retry)
  B| SELECT stock, version FROM products WHERE id = 1  -> (9, 2)
  B| UPDATE products SET stock = 8, version = version + 1 WHERE id = 1 AND version = 2  -> UPDATE 1
  M| SELECT stock, version FROM products WHERE id = 1  -> (8, 3)
```

B's first update matched **0 rows** — the conflict signal (the version is no longer the *expected* 1). B re-reads (9, version 2) and succeeds. The database checks and writes in one atomic step; `UPDATE 0` means "someone else won — reload and retry, or tell the user the record changed". JPA/Hibernate's `@Version`, Rails' `lock_version`, DynamoDB conditional writes and Redis `WATCH` / `MULTI` / `EXEC` are the same idea: no locks (so no deadlocks), great when conflicts are rare, wasteful retries when they are frequent.

### Comparison

| Fix | Blocking? | Retry needed? | Best for |
|---|---|---|---|
| Atomic `UPDATE … SET x = x - 1` | Briefly (row lock) | No | Counters, balances — anything expressible in SQL |
| `SELECT … FOR UPDATE` | Yes, until commit | No | Complex logic between read and write, short transactions |
| Repeatable Read / Serializable | No (aborts instead) | Yes (40001) | Code that already has a retry loop |
| Version column (CAS) | No | Yes (0 rows) | Long "think time", web forms, APIs, distributed caches |

### In the interview

> **30-second answer:** "Two read-modify-write cycles interleave and one write overwrites the other. Fixes: atomic SQL arithmetic; SELECT FOR UPDATE; Repeatable Read, where Postgres aborts the second writer (40001); or a version column where zero updated rows means retry."

- *Which would you use for a "like" counter?* Atomic `UPDATE posts SET likes = likes + 1` (or a separate likes table and count).
- *Which for a user editing a product page for minutes?* Version column — never hold a DB lock across user think time.
- *Does Read Committed prevent lost updates?* Only if the write is expressed as one atomic statement; not for app-side read-modify-write.

### Self-check

1. Why did the naive version end at 9? → Both read 10 and both wrote 10 − 1.
2. What does `UPDATE 0` mean in optimistic locking? → The version changed; someone else updated first.
3. What SQLSTATE does PostgreSQL use for a serialization failure? → 40001.

---

## 13.8 Write skew and Serializable isolation

### The problem

Hospital rule: **at least one doctor on call for the night shift**. Dr Rao and Dr Kim, both on call, each click "go off duty"; the app checks "is anyone else on call?" and, if so, updates *that doctor's own row*. Different rows, so no row-lock conflict — yet the rule can break.

#### Repeatable Read (snapshot isolation) allows it

```
=== Write skew: REPEATABLE READ lets both doctors go off duty ===
  A| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  B| BEGIN ISOLATION LEVEL REPEATABLE READ  -> BEGIN
  A| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  B| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  B| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_kim'  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| COMMIT  -> COMMIT
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 0
```

**Nobody** is on call. Each transaction was correct against its snapshot, but run one after the other the second would have seen 1 and refused. The same shape appears in double-booking a room, claiming a username checked by

`SELECT` first, or overspending a shared budget.

### Serializable (SSI) aborts one

```
=== Write skew: SERIALIZABLE aborts one of them ===
  A| BEGIN ISOLATION LEVEL SERIALIZABLE  -> BEGIN
  B| BEGIN ISOLATION LEVEL SERIALIZABLE  -> BEGIN
  A| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  B| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 2
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  B| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_kim'  -> UPDATE 1
  A| COMMIT  -> COMMIT
  B| COMMIT  -> ERROR 40001: could not serialize access due to read/write dependencies among transactions
     DETAIL: Reason code: Canceled on identification as a pivot, during commit attempt.
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 1
```

How **Serializable Snapshot Isolation (SSI)** caught it, in steps:

1. Each Serializable transaction still reads from a snapshot (readers and writers don't block).
2. It records what it **read** as *SIREAD* "locks" — non-blocking notes such as "B read the night-shift rows".
3. When a transaction writes something a concurrent one read, a **read-write dependency** (rw-conflict) is recorded: here "A wrote what B read" and "B wrote what A read".
4. Such a cycle means no serial order could produce the result. On the dangerous pattern (a "pivot" transaction with a conflict in and a conflict out) PostgreSQL aborts one transaction with **40001**; the retried B sees only 1 doctor on call and refuses.

The error can appear at any statement, including `COMMIT`, so **every Serializable transaction needs a retry loop**:

```python
for attempt in range(5):
    try:
        with conn.transaction():                        # BEGIN ... COMMIT
            conn.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
            go_off_duty(conn, doctor)
        break
    except psycopg.errors.SerializationFailure:         # SQLSTATE 40001
        continue                                        # rerun the whole transaction
```

SSI can also abort harmless transactions (false positives) — its bookkeeping is approximate and coarsens from rows to pages or whole tables when a transaction reads a lot. Keep Serializable transactions short and mark read-only ones `READ ONLY`.

### Fix without Serializable: lock what you based the decision on

If you know exactly which rows the decision depends on, lock them. Here both transactions run at the default Read Committed:

```
=== Write skew fix without SERIALIZABLE: lock the rows you base the decision on ===
  A| BEGIN  -> BEGIN
  B| BEGIN  -> BEGIN
  A| SELECT doctor FROM on_call WHERE shift = 'night' AND on_duty FOR UPDATE  -> (dr_rao), (dr_kim)
  B| SELECT doctor FROM on_call WHERE shift = 'night' AND on_duty FOR UPDATE
     .. B is WAITING (wait_event=transactionid, blocked by A)
  A| UPDATE on_call SET on_duty = false WHERE doctor = 'dr_rao'  -> UPDATE 1
  A| COMMIT  -> COMMIT
     -> (B resumes) dr_kim
  (B's app sees only one doctor left, so it refuses to go off duty)
  B| ROLLBACK  -> ROLLBACK
  M| SELECT count(*) FROM on_call WHERE shift = 'night' AND on_duty  -> 1
```

After waiting, B's `FOR UPDATE` re-checks the rows and returns only Dr Kim. Limitation: `FOR UPDATE` locks only rows that **exist**. If the rule is about the *absence* of rows ("no booking overlaps 10:00–11:00"), there is

nothing to lock. Then use Serializable; a constraint (`UNIQUE`, or PostgreSQL's exclusion constraint `EXCLUDE USING gist (room WITH =, during WITH &&)`, which needs the `btree_gist` extension for the `=` part); or *materialize the conflict* — pre-create lockable rows, e.g., one per room per time slot.

### In the interview

> **30-second answer:** "Two transactions read the same data, decide on it, and write different rows — no write conflict, but an invariant breaks. Snapshot isolation allows it. Fixes: Serializable (SSI aborts one with 40001 — retry), FOR UPDATE on the rows the decision depends on, or a unique/exclusion constraint."

- *Why doesn't FOR UPDATE fix double-booking of an empty slot?* The conflicting row doesn't exist yet (a phantom); use a constraint, Serializable, or materialized slot rows.

### Self-check

1. At Repeatable Read, why was no conflict detected? → The two transactions updated different rows.
2. What are SIREAD locks? → Non-blocking records of what a Serializable transaction read, used to find rw-dependencies.
3. Name two non-Serializable fixes. → `FOR UPDATE` on the rows read; a unique/exclusion constraint.

---

## 13.9 Concurrency control — the big picture

### What it is

**Concurrency control** is the part of a database that decides how overlapping transactions may interleave so the result stays correct: no anomalies from §13.4 (data integrity), each transaction isolated to the degree its level promises, and as much parallelism as possible (performance).

The formal correctness goal is **serializability**: a concurrent execution (a *schedule*) is correct if its effect equals that of *some* serial order of the same transactions. Two operations **conflict** if they belong to different transactions, touch the same item, and at least one is a write; a schedule is *conflict-serializable* if its conflict graph has no cycle.

#### The four families

| Technique | Core idea | Waits or aborts? | Used by |
|---|---|---|---|
| **Pessimistic / lock-based** (e.g., 2PL) | Lock before you touch; others wait | Waits (deadlocks possible) | SQL Server, MySQL locking reads, DB2; `FOR UPDATE` everywhere |
| **Optimistic (OCC)** | Work freely, validate at commit, abort on conflict | Aborts | App-level version columns, some in-memory and distributed DBs |
| **Timestamp ordering** | Each txn gets a timestamp; operations must respect timestamp order | Aborts | Mostly textbook; ideas used in distributed DBs |
| **MVCC** | Keep several versions; readers read an old version, never block writers | Readers never wait; writers wait for writers | PostgreSQL, Oracle, MySQL InnoDB, SQL Server snapshot modes |

These are combined in practice: PostgreSQL = MVCC for reads + row locks for writes + SSI (optimistic validation) for Serializable.

#### Optimistic vs pessimistic

**Optimistic concurrency control (OCC)** assumes conflicts are **rare**. A transaction runs in three phases: (1) **read/execute** in a private workspace, taking no locks; (2) **validate** at commit — did a transaction that committed meanwhile change anything it read? (3) **write** if not, otherwise **abort and retry**.

*Example:* users A and B open the same product; A saves first; B's save fails validation and retries — the version-column pattern of §13.7.

**Pessimistic concurrency control (PCC)** assumes conflicts are **likely**: **lock** data when first touching it, make conflicting transactions **wait**, and **release** at commit or rollback.

*Example:* A locks the product row; B waits until A finishes (§13.7 Fix 2).

| Aspect | Optimistic (OCC) | Pessimistic (PCC) |
|---|---|---|
| Assumes | Conflicts are rare | Conflicts are common |
| Locks | None during execution | Taken on access, held to the end |
| Cost of a conflict | Abort and redo the work | Waiting |
| Overhead with no conflicts | Very low | Lock management on every access |
| Deadlocks | None (no waits) | Possible — need detection |
| Fits | Read-heavy, low-contention, distributed systems, long user think time | Write-heavy, hot rows (a popular product's stock), critical updates where retrying is expensive |

`Real-world examples.` **Git** is optimistic: nobody locks a file, and conflicts surface only at push/merge time. **Google Docs** lets everyone type and reconciles edits as they sync. **Banking** and **ticketing** lean pessimistic on hot rows so a seat is never sold twice; **e-commerce** locks (or atomically decrements) the stock row at purchase.

### In the interview

> **30-second answer:** "Keep interleaved transactions serializable. Pessimistic: lock and wait (high contention; deadlocks possible). Optimistic: validate at commit and abort on conflict (rare conflicts). Timestamp ordering: follow start order. MVCC: versions so readers never block writers. Postgres combines MVCC, row locks and SSI."

### Self-check

1. Three phases of OCC? → Read/execute, validate, write (or abort).
2. Why does OCC have no deadlocks? → Transactions never wait for each other's locks.
3. Which is better for a flash sale on one product? → Pessimistic (or an atomic update): contention is very high, OCC would abort constantly.

---

## 13.10 Lock-based concurrency control

### What it is

With **lock-based concurrency control**, a transaction must obtain a **lock** on data before using it; a conflicting request waits until the lock is released — so others can't read or overwrite your in-progress changes.

## Types of locks

| Lock | Purpose | Others may… |
|---|---|---|
| **Shared (S)** | Reading | also take S (read together); may **not** write |
| **Exclusive (X)** | Writing | do nothing with the item (no S, no X) until released |
| **Intention Shared (IS)** | On a table: "I will take S locks on some rows inside" | – |
| **Intention Exclusive (IX)** | On a table: "I will take X locks on some rows inside" | – |

## Lock granularity

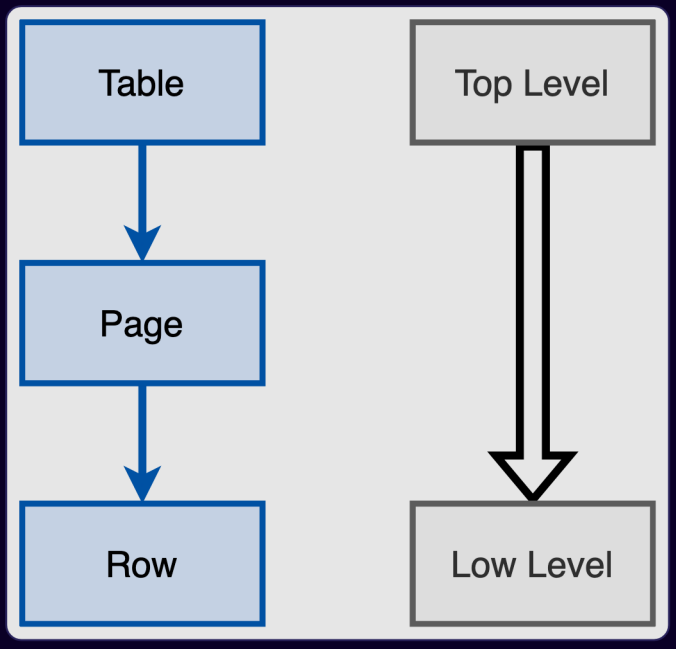



*Figure – locks can be taken on a whole table, a page or a single row (course diagram).*

| Granularity | Advantage | Disadvantage | Example |
|---|---|---|---|
| **Row** | Maximum concurrency – other rows stay free | Many locks to manage (memory, CPU) | Update one customer's row |
| **Page** | Middle ground | Locks unrelated rows that share the page | Some storage engines (older SQL Server, DB2) |
| **Table** | Simple, very low overhead | Blocks everyone else | Batch update of the whole `orders` table |

Some engines **escalate**: when a transaction holds thousands of row locks, they are swapped for one table lock (SQL Server does this). PostgreSQL never escalates row locks because it stores them in the rows themselves.

## Intention locks – why they exist

T1 holds an X lock on row 5 of `employees`; T2 asks for an S lock on the **whole table**. Without extra information the lock manager would have to scan every row lock for conflicts. **Intention locks** make locking *hierarchical*:

1. T1 first places **IX** on the `employees` table, then **X** on row 5.
2. T2 asks for **S** on the table. The manager only checks table-level locks: S conflicts with IX, so T2 waits – no row scan needed.
3. A T3 wanting X on row 9 places IX on the table (IX is compatible with IX) and proceeds in parallel with T1.

Compatibility (can the requested lock be granted while another transaction holds…?):

| Held → / Requested ↓ | IS | IX | S | X |
|---|---|---|---|---|
| IS | yes | yes | yes | no |
| IX | yes | yes | no | no |
| S | yes | no | yes | no |
| X | no | no | no | no |

### Two-phase locking (2PL)

**2PL** is the classic protocol that guarantees conflict-serializability, in two phases:

1. **Growing (expanding) phase** – it may acquire locks but not release any.
2. **Shrinking phase** – once it releases its first lock, it may not acquire any new one.

The moment it holds the most locks is the **lock point**; transactions are serialized in the order of their lock points.

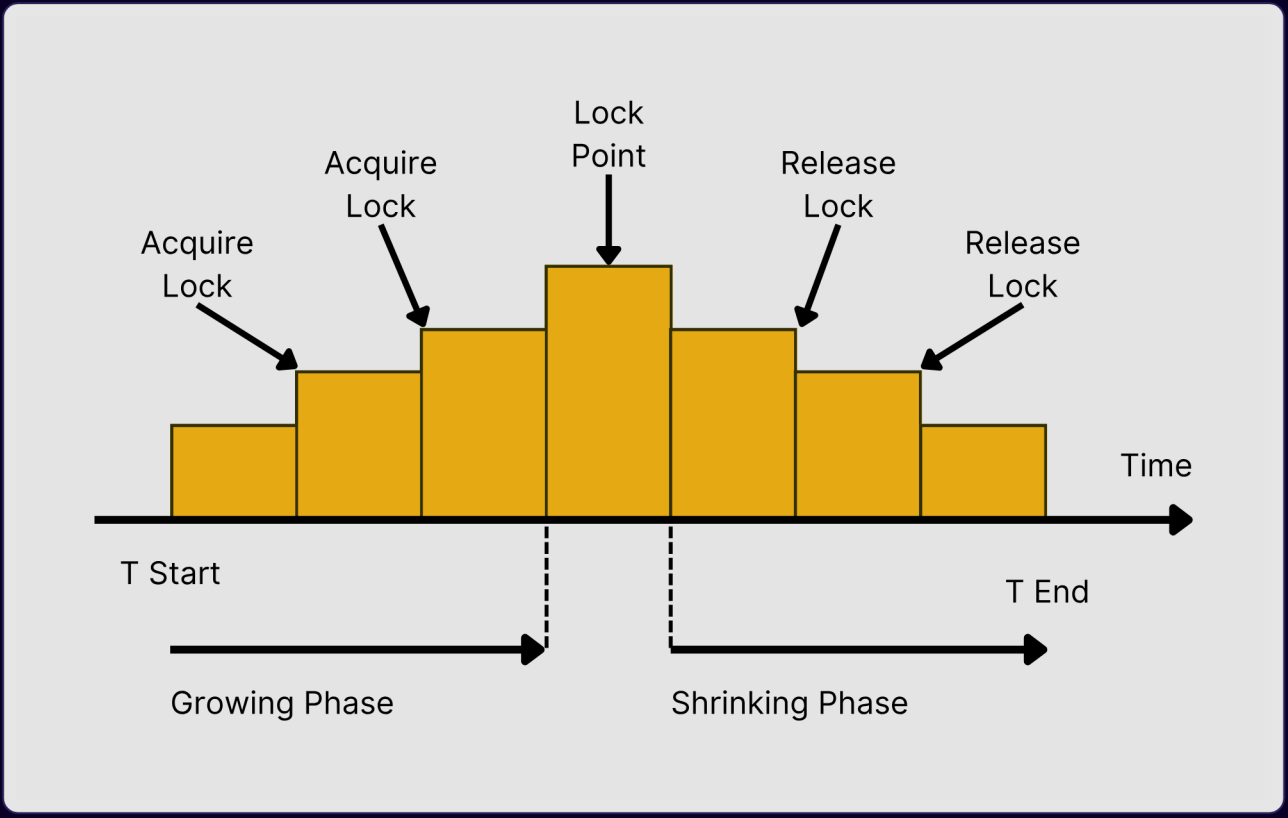



*Figure – the number of locks held rises, peaks at the lock point, then falls (course diagram).*

Variants you should know:

| Variant | Rule | Why |
|---|---|---|
| Basic 2PL | grow, then shrink | Serializable, but a transaction could read data from another that later aborts (*cascading aborts*) |
| **Strict 2PL** | hold all **X** locks until commit/abort | No one reads uncommitted data; what most lock-based databases do |
| **Rigorous 2PL** | hold **all** locks until commit/abort | Simplest to reason about; serial order = commit order |
| Conservative 2PL | take every lock before starting | No deadlocks, but you must know the data set in advance |

*Pros:* guaranteed serializability. *Cons:* **deadlocks** (§13.11) and lower throughput under contention – a long reader blocks writers and vice versa. MySQL and SQL Server implement Serializable with (strict) 2PL; it suits strong-isolation domains such as finance.

## Predicate locking and index-range locking

2PL on existing rows can't stop **phantoms**: a new row that *would* match your query didn't exist when you locked. Two answers:

**Predicate locking** extends 2PL by locking the **condition** itself — every row, existing or future, that satisfies `WHERE …`. Example: table `seats` holds seat numbers 10 and 30; transaction T runs `UPDATE seats SET held = true WHERE seat_no BETWEEN 10 AND 30`.

| Seat | Exists now? | Locked by predicate lock? |
|---|---|---|
| 10 | yes | yes |
| 20 | no | yes — an insert of seat 20 must wait |
| 30 | yes | yes |

It prevents phantoms and write skew, but checking every lock request against every active predicate is expensive, and it over-locks.

**Index-range locking** (*next-key* or *gap* locking) approximates the predicate with ranges of an **index**: it locks the matching index entries *and the gaps between them*, so an insert of seat 20 must wait. (Locking only the existing keys 10 and 30 would let seat 20 in — phantoms would remain; covering the gap is the whole point.) It is cheaper than predicate locking because it reuses the index, at the price of sometimes locking a slightly wider range; without a suitable index the engine locks far more (up to the whole table). MySQL InnoDB's next-key locks at Repeatable Read work this way.

| Aspect | 2PL on rows | Predicate locking | Index-range (next-key) locking |
|---|---|---|---|
| What is locked | Rows actually touched | Every row matching the condition, even future ones | Matching index entries plus gaps between them |
| Phantoms prevented? | No | Yes | Yes (needs an index on the condition) |
| Seen in | All lock-based engines | Theory; PostgreSQL SSI's SIREAD locks are a non-blocking cousin | MySQL InnoDB, SQL Server key-range locks |

## PostgreSQL's locks in practice

PostgreSQL has eight **table-level lock modes** (taken automatically by statements) and four **row-level** modes. `code/04_lock_modes.sql` shows what a transaction holds:

```
BEGIN;
SELECT * FROM accounts WHERE id = 1;                        -- plain read
UPDATE products SET stock = stock - 1 WHERE id = 1;         -- write
SELECT * FROM on_call WHERE doctor = 'dr_rao' FOR SHARE;    -- shared row lock
LOCK TABLE jobs IN SHARE MODE;                              -- explicit table lock
SELECT c.relname, l.mode, l.granted
FROM pg_locks l JOIN pg_class c ON c.oid = l.relation
WHERE l.pid = pg_backend_pid() AND c.relnamespace = 'public'::regnamespace
ORDER BY c.relname, l.mode;
COMMIT;

SELECT pg_try_advisory_lock(42) AS got_it_first_time;
SELECT count(*) AS advisory_locks_held FROM pg_locks WHERE locktype = 'advisory' AND pid = pg_backend_pid();
SELECT pg_advisory_unlock(42) AS released;
```

Output (row echoes trimmed; the three advisory-lock results condensed onto one line):

```
    relname    |       mode       | granted
---------------+------------------+---------
 accounts      | AccessShareLock  | t
 accounts_pkey | AccessShareLock  | t
 jobs          | ShareLock        | t
 on_call       | RowShareLock     | t
 on_call_pkey  | RowShareLock     | t
 products      | RowExclusiveLock | t
 products_pkey | RowExclusiveLock | t
(7 rows)
...
 got_it_first_time → t     advisory_locks_held → 1     released → t
```

These table modes play the role of intention locks: `RowExclusiveLock` (taken by `UPDATE/INSERT/DELETE`) means "I'm changing rows in here"; it doesn't conflict with other writers but does conflict with `SHARE` and stronger modes:

| Common statement | Table lock taken | Blocks |
|---|---|---|
| `SELECT` | ACCESS SHARE | only ACCESS EXCLUSIVE (`DROP`, most `ALTER TABLE`) |
| `SELECT … FOR UPDATE/SHARE` | ROW SHARE | EXCLUSIVE, ACCESS EXCLUSIVE |
| `INSERT/UPDATE/DELETE` | ROW EXCLUSIVE | SHARE and stronger |
| `CREATE INDEX` | SHARE | writers |
| `CREATE INDEX CONCURRENTLY`, `VACUUM` | SHARE UPDATE EXCLUSIVE | other schema changes / vacuums |
| most `ALTER TABLE`, `DROP`, `TRUNCATE`, `VACUUM FULL` | ACCESS EXCLUSIVE | everything, even `SELECT` |

**Row-level lock modes**, strongest first: `FOR UPDATE` (taken by `DELETE` or an update of key columns), `FOR NO KEY UPDATE` (a normal `UPDATE`), `FOR SHARE` (read, block changes), `FOR KEY SHARE` (foreign-key checks; blocks only key changes/deletes). **Advisory locks** (`pg_advisory_lock(n)`) lock a number that only your application gives meaning to — e.g., "only one nightly job may run".

## Common mistakes

- An `ALTER TABLE` on a busy table waits for ACCESS EXCLUSIVE — and **every** later query queues behind it. Use `SET lock_timeout = '2s'` for migrations.
- Holding locks during slow work (HTTP calls, user input).
- Relying on the absence of a row without a unique constraint or Serializable (phantoms).

## In the interview

> **30-second answer:** "S locks for reads, X for writes; only S is compatible with S. Finer granularity means more concurrency and more overhead; intention locks let the manager check table-versus-row conflicts without scanning rows. 2PL (grow then shrink) guarantees serializability, strict 2PL keeps write locks to commit, and next-key locks stop phantoms."

- *Why does strict 2PL hold X locks until commit?* So nobody reads uncommitted data and cascading aborts can't happen.
- *How does Postgres avoid lock-table explosion for row locks?* Row locks are stored in the tuple header (`xmax` + infomask); waiters wait on the holder's transaction ID.

## Self-check

1. Can two transactions hold S on the same row? X? → S yes; X no.
2. What does IX on a table announce? → The transaction will take X locks on some rows in it.

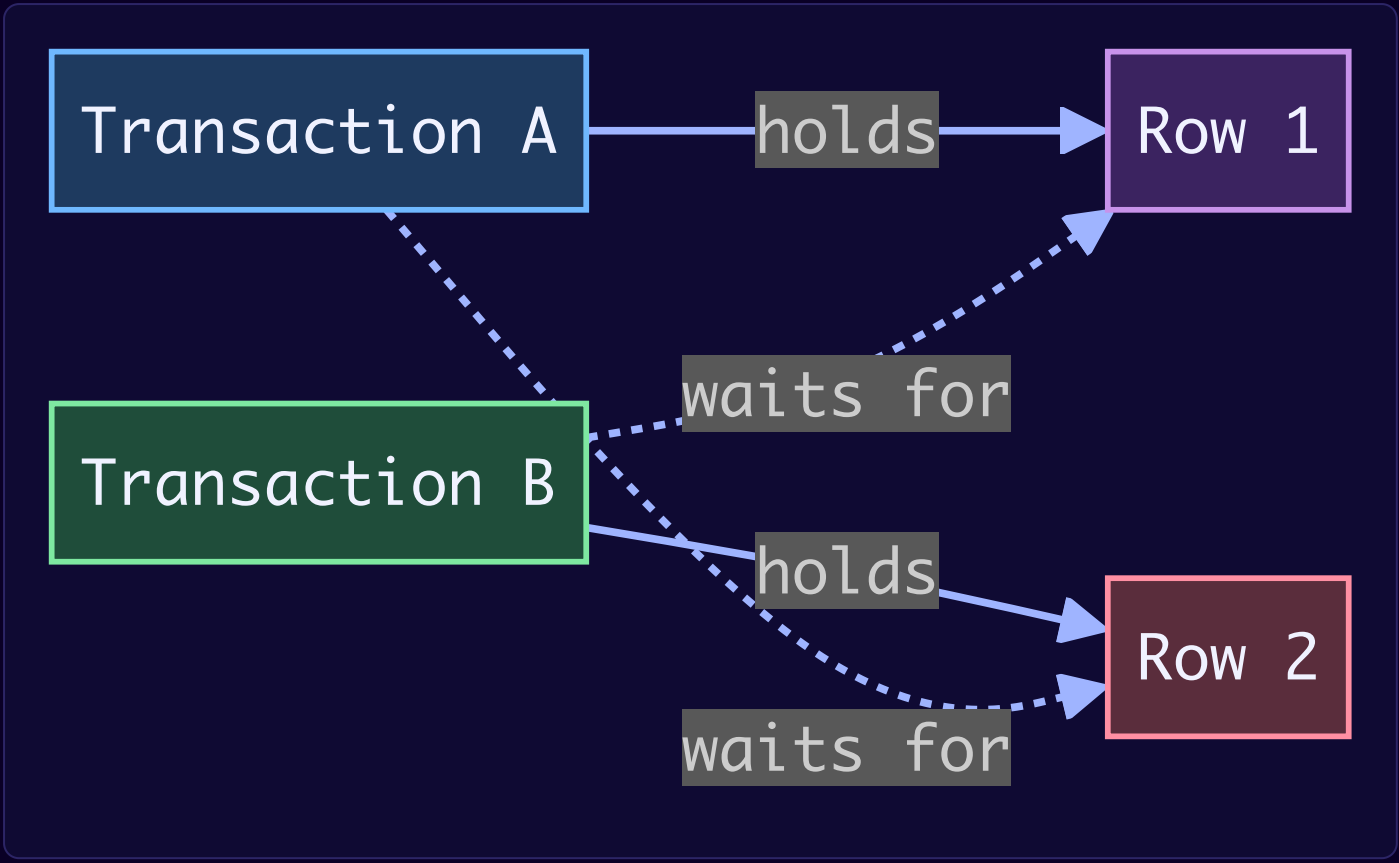
3. How does index-range locking stop phantoms? → It locks the gaps between index keys, so matching inserts wait.

---

## 13.11 Deadlocks

### What it is

A **deadlock** is a cycle of waiting: T1 holds resource 1 and waits for resource 2, while T2 holds 2 and waits for 1. Neither can ever proceed unless something intervenes.




### How databases handle it

| Strategy | How | Trade-off |
|---|---|---|
| **Detection** | Build a *wait-for graph* (edge T1 → T2 when T1 waits for T2); a cycle is a deadlock; abort one victim | Some work lost; used by PostgreSQL, MySQL, SQL Server, Oracle |
| **Timeout** | Abort any transaction that waits too long | Simple, but may abort non-deadlocked waiters or react slowly (`lock_timeout`, InnoDB `innodb_lock_wait_timeout`) |
| **Prevention by ordering** | Always lock resources in the same global order (e.g., lower `id` first) | Needs discipline in application code — the most practical fix |
| **Wait-die / wound-wait** | By start timestamp: in *wait-die* an older requester waits, a younger one aborts ("dies"); in *wound-wait* an older requester aborts ("wounds") the younger holder, a younger one waits | No cycles; more aborts; some distributed databases |
| **Conservative 2PL** | Take all locks up front | Must know everything in advance |

### Seeing PostgreSQL detect one

Two transfers in opposite directions: A moves 10 from account 1 to 2; B moves 10 from 2 to 1. Each locks its "from" row first.

```
=== Deadlock: PostgreSQL detects it and aborts one transaction ===
  A| SET deadlock_timeout = '10s'  -> SET
  B| SET deadlock_timeout = '200ms'  -> SET
  A| BEGIN  -> BEGIN
  B| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = balance - 10 WHERE id = 1  -> UPDATE 1
  B| UPDATE accounts SET balance = balance - 10 WHERE id = 2  -> UPDATE 1
  A| UPDATE accounts SET balance = balance + 10 WHERE id = 2
     .. A is WAITING (wait_event=transactionid, blocked by B)
  B| UPDATE accounts SET balance = balance + 10 WHERE id = 1  -> ERROR 40P01: deadlock detected
     DETAIL: Process <B> waits for ShareLock on transaction <xid>; blocked by process <A>.
             Process <A> waits for ShareLock on transaction <xid>; blocked by process <B>.
  B| ROLLBACK  -> ROLLBACK
     -> (A resumes) UPDATE 1
  A| COMMIT  -> COMMIT
  M| SELECT id, balance FROM accounts WHERE id IN (1, 2) ORDER BY id  -> (1, 490.00), (2, 310.00)
```

How PostgreSQL does it:

1. A waiting backend first sleeps for `deadlock_timeout` (default **1 s**) – most waits end by themselves, and the check is costly.
2. Then it inspects the wait-for graph. B's timer fired first (the demo set 200 ms for B, 10 s for A to make the victim predictable); it found the cycle A → B → A and aborted **B** with SQLSTATE **40P01**.
3. B's locks are released, A's update completes and commits; the application retries B.

**The fix** for this pattern is ordering: always lock the lower account id first (`SELECT … FROM accounts WHERE id IN (1, 2) ORDER BY id FOR UPDATE`), so both transfers queue for row 1 and no cycle can form.

### Common mistakes

- Not retrying on 40P01 (or 40001) – both mean "try the whole transaction again".
- Locking rows in input order (an unsorted `for item in cart` loop) – sort by primary key first.

### In the interview

> **30-second answer:** "A cycle in the wait-for graph. Databases detect it (Postgres after `deadlock_timeout`, 1 s by default) and abort a victim, which the app retries. Prevent by locking in a consistent order and keeping transactions short; textbook alternatives are wait-die and wound-wait."

### Self-check

1. What graph is used for detection? → The wait-for graph between transactions.
2. PostgreSQL's deadlock SQLSTATE? → 40P01.
3. Simplest prevention in application code? → Acquire locks in a fixed global order.

---

## 13.12 Row locks for work queues: `FOR UPDATE`, `NOWAIT`, `SKIP LOCKED`

### The problem it solves

Many apps keep a **job table** (emails to send, images to resize) served by several workers, each of which must grab jobs **no other worker** holds. With plain `FOR UPDATE` all workers lock the *same* first rows and queue behind each other.

### The options

| Clause | If the row is locked by someone else… |
|---|---|
| `FOR UPDATE` | wait until they finish |
| `FOR UPDATE NOWAIT` | fail immediately with error 55P03 |
| `FOR UPDATE SKIP LOCKED` | silently skip that row and take the next unlocked one |

`SKIP LOCKED` exists in PostgreSQL (9.5+), MySQL (8.0+) and Oracle; SQL Server's `READPAST` hint does the same.

### Seeing it work

```
=== Job queue: FOR UPDATE SKIP LOCKED (and NOWAIT) ===
  A| BEGIN  -> BEGIN
  A| SELECT id, payload FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED  -> (1,
email #1), (2, email #2)
  B| BEGIN  -> BEGIN
  B| SELECT id, payload FROM jobs WHERE status = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED  -> (3,
email #3), (4, email #4)
  B| SELECT id FROM jobs WHERE id = 1 FOR UPDATE NOWAIT  -> ERROR 55P03: could not obtain lock on row in
relation "jobs"
  B| ROLLBACK  -> ROLLBACK
  B| UPDATE jobs SET status = 'running', claimed_by = 'B' WHERE id IN (SELECT id FROM jobs WHERE status =
'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED) RETURNING id  -> (3), (4)
  A| UPDATE jobs SET status = 'done', claimed_by = 'A' WHERE id IN (1, 2)  -> UPDATE 2
  A| COMMIT  -> COMMIT
  M| SELECT id, status, claimed_by FROM jobs ORDER BY id  -> (1, done, A), (2, done, A), (3, running, B), (4,
running, B), (5, queued, None), (6, queued, None)
```

Worker A locked jobs 1–2; worker B, running the identical query, got 3–4 **without waiting**, and `NOWAIT` on job 1 failed at once. The usual form **claims and marks in one statement**:

```
UPDATE jobs SET status = 'running', claimed_by = $1
WHERE id IN (SELECT id FROM jobs
             WHERE status = 'queued'
             ORDER BY id
             LIMIT 10
             FOR UPDATE SKIP LOCKED)
RETURNING id, payload;
```

Two designs: (a) keep the transaction open while processing — a crashed worker's rollback releases its jobs automatically; (b) mark `running` and commit at once, plus a `claimed_at` column and a sweeper that re-queues stuck jobs. A partial index `CREATE INDEX ON jobs (id) WHERE status = 'queued'` keeps the claim fast (chapter 12).

## When not to use it

`SKIP LOCKED` deliberately hides locked rows, so never use it for reports or business-rule checks. For very high throughput, fan-out or replay, use a broker (Kafka, RabbitMQ, SQS); a Postgres queue shines when jobs must be created in the same transaction as the business data.

## Real-world examples

Job-queue libraries such as GoodJob (Ruby), Oban (Elixir), graphile-worker (Node) and Procrastinate (Python) use this pattern; ticketing systems use it to hand concurrent buyers different free seats.

---

# 13.13 Timestamp-based concurrency control

## What it is

**Timestamp ordering (TO)** gives every transaction a unique, increasing **timestamp** TS(T) at start (clock or counter). The result must equal running the transactions **in timestamp order**, older first; instead of waiting for locks, a transaction that would break the order is **aborted and restarted** with a new timestamp. The result is always serializable.

### Key bookkeeping

| Symbol | Kept for | Meaning |
|---|---|---|
| **TS(T)** | each transaction | its start timestamp – its place in the serial order |
| **RTS(X)** | each data item X | the largest timestamp of any transaction that has **read** X |
| **WTS(X)** | each data item X | the largest timestamp of any transaction that has **written** X |

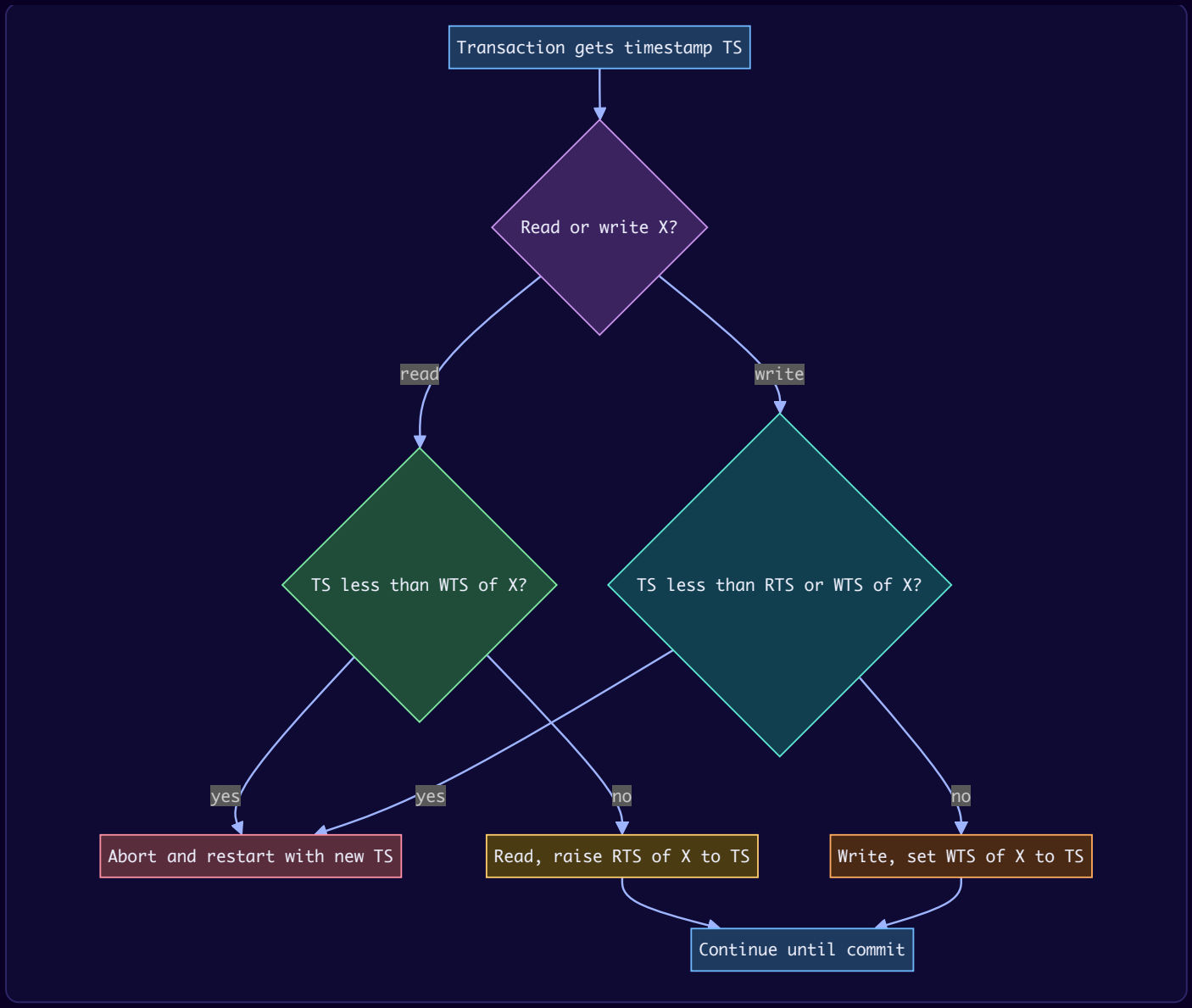
## How it works – the rules

**When T wants to read X:**

1. If TS(T) < WTS(X): a *younger* transaction already overwrote X – T would read a value "from its future". **Abort T** (restart with a new timestamp).
2. Otherwise read X and set RTS(X) = max(RTS(X), TS(T)).

**When T wants to write X:**

1. If TS(T) < RTS(X): a younger transaction already read the old value. **Abort T.**
2. If TS(T) < WTS(X): a younger transaction already wrote X, so T's write is outdated. **Abort T** – or, under the **Thomas write rule**, **skip** the obsolete write (timestamp order would have overwritten it anyway).
3. Otherwise write X and set WTS(X) = TS(T).




## Worked example — booking the last seats

Item S = seats left for a show (initially RTS = 0, WTS = 0). Priya's booking transaction starts first and gets TS = 5; Omar's starts a moment later with TS = 7.

| Step | Operation | Check | Result | RTS(S) | WTS(S) |
|---|---|---|---|---|---|
| 1 | T5 reads S | 5 ≥ WTS 0 | allowed | 5 | 0 |
| 2 | T7 reads S | 7 ≥ WTS 0 | allowed | 7 | 0 |
| 3 | T7 writes S (books a seat) | 7 ≥ RTS 7 and ≥ WTS 0 | allowed | 7 | 7 |
| 4 | T5 writes S (books a seat) | 5 < RTS 7 | **T5 aborted** | 7 | 7 |
| 5 | T5 restarts as T9, reads S | 9 ≥ WTS 7 | allowed, sees Omar's booking | 9 | 7 |

Priya's first attempt relied on a count that a "later" transaction had already read and changed, so it was rolled back instead of double-selling; the retry sees the current count.

### Trade-offs

| Advantages | Disadvantages |
|---|---|
| **No deadlocks** – nobody waits for a lock | **Many rollbacks** under contention; work is thrown away |
| Guarantees **serializability** by construction | **Starvation** – a long transaction may be restarted again and again because younger ones keep touching its data |
| Simple conflict rule: compare two numbers | **Overhead** – every read updates RTS (reads become metadata writes); extra care is needed to avoid reading uncommitted values |

### Where it is used

Pure TO is mostly a textbook protocol. Its descendant **multiversion timestamp ordering** (versions tagged with timestamps; each read picks the right one) underlies many MVCC systems, and distributed databases rely on timestamps: Google Spanner uses TrueTime, CockroachDB hybrid logical clocks plus MVCC.

### In the interview

> **30-second answer:** "Transactions must behave as if run in start-timestamp order. Each item keeps its largest read and write timestamps; a read is rejected if a younger transaction already wrote the item, a write if a younger one already read or wrote it, and the loser restarts with a new timestamp. No deadlocks, but many aborts and possible starvation; the Thomas write rule skips obsolete writes."

### Self-check

1. T with TS 4 wants to write X where RTS(X) = 6. → Abort T (a younger transaction read X).
2. What does the Thomas write rule change? → An outdated write (TS < WTS, but ≥ RTS) is ignored instead of causing an abort.
3. Why can't TO deadlock? → Transactions never wait; conflicts cause aborts.

---

## 13.14 Multiversion concurrency control (MVCC)

### What it is

**MVCC** keeps **several versions** of each row. A write creates a **new version** and marks the old one as replaced; each transaction reads from a **snapshot** – the versions committed as of a certain moment. So **readers and writers never block each other** (writers still block writers on the same row), and every transaction sees a consistent picture.

*Analogy:* a shared document with full version history. Editing creates a new revision; someone reading "as of 10:00" keeps reading that revision undisturbed.

### How it works

1. **Versions carry metadata:** the transaction that **created** it and the one that **deleted/replaced** it (empty if current).
2. **Snapshots:** when a transaction (Repeatable Read) or statement (Read Committed) starts, the database notes which transactions had committed.
3. **Visibility:** a version is visible if its creator committed before the snapshot and its deleter is empty, aborted, or not committed as of the snapshot.
4. **Writes** create new versions; concurrent writers of one row are serialized by a row lock (§13.7), and at Repeatable Read/Serializable a concurrent committed change raises a serialization error.
5. **Cleanup:** versions no snapshot can see are garbage (PostgreSQL's `VACUUM`, InnoDB's purge thread).

### Worked example — versions of an account

Account A starts at 500 (created by transaction 1); account B at 300. Transaction 2 adds 200 to A; later, transaction 3 subtracts 100 from A.

| After | Versions of account A (created_by, deleted_by, balance) | Account B |
|---|---|---|
| txn 1 | (1, –, 500) | (1, –, 300) |
| txn 2 | (1, 2, 500) · (2, –, 700) | unchanged |
| txn 3 | (1, 2, 500) · (2, 3, 700) · (3, –, 600) | unchanged |

A reader whose snapshot predates txn 2's commit still sees 500; one taken between txn 2 and txn 3 sees 700; new readers see 600 — simultaneously, without locks.

### MVCC in PostgreSQL: `xmin` and `xmax` for real

PostgreSQL stores the "created by" and "deleted by" fields in every row version as hidden system columns: `xmin` (inserting transaction ID) and `xmax` (deleting/locking transaction ID, 0 if none). `ctid` is the version's physical address (page, slot).

**One session** — `code/03_mvcc_xmin_xmax.sql` repeats the worked example (500 → +200 → −100) and then uses the `pageinspect` extension to look *inside* the table's first 8 KB page:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;
CREATE TABLE wallet (id int PRIMARY KEY, balance int);
INSERT INTO wallet VALUES (1, 500), (2, 300);
SELECT ctid, xmin, xmax, id, balance FROM wallet ORDER BY id;

UPDATE wallet SET balance = balance + 200 WHERE id = 1;
UPDATE wallet SET balance = balance - 100 WHERE id = 1;
SELECT ctid, xmin, xmax, id, balance FROM wallet ORDER BY id;

SELECT lp AS slot, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('wallet', 0)) ORDER BY lp;

BEGIN;
UPDATE wallet SET balance = 0 WHERE id = 2;
SELECT txid_current() AS my_txid;
ROLLBACK;
SELECT ctid, xmin, xmax, id, balance FROM wallet WHERE id = 2;

VACUUM wallet;
SELECT lp AS slot, lp_flags, t_xmin, t_xmax, t_ctid
FROM heap_page_items(get_raw_page('wallet', 0)) ORDER BY lp;
```

Output (transaction IDs differ on every run; command tags trimmed):

```
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,1) |  802 |    0 |  1 |     500
 (0,2) |  802 |    0 |  2 |     300
(2 rows)

UPDATE 1
UPDATE 1
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,4) |  804 |    0 |  1 |     600
 (0,2) |  802 |    0 |  2 |     300
(2 rows)

 slot | t_xmin | t_xmax | t_ctid
------+--------+--------+--------
    1 |    802 |    803 | (0,3)
    2 |    802 |      0 | (0,2)
    3 |    803 |    804 | (0,4)
    4 |    804 |      0 | (0,4)
(4 rows)

BEGIN
UPDATE 1
 my_txid
---------
     805
(1 row)

ROLLBACK
 ctid  | xmin | xmax | id | balance
-------+------+------+----+---------
 (0,2) |  802 |  805 |  2 |     300
(1 row)

VACUUM
 slot | lp_flags | t_xmin | t_xmax | t_ctid
------+----------+--------+--------+--------
    1 |        2 |        |        |
    2 |        1 |    802 |    805 | (0,5)
    3 |        0 |        |        |
    4 |        1 |    804 |      0 | (0,4)
    5 |        0 |        |        |
(5 rows)
```

Reading it:

1. Transaction 802 inserted both rows ( `xmax = 0` : not replaced).
2. After the updates, `SELECT` shows only the current version of row 1, at a **new address** `(0,4)` with `xmin = 804` : an update is "insert a new version + mark the old one".
3. The raw page still holds **all three versions** of row 1 — the worked example exactly: slot 1 (500, created by 802, replaced by 803), slot 3 (700, 803 → 804), slot 4 (600, current). Each old version's `t_ctid` points to its successor.
4. The rolled-back update left `xmax = 805` on row 2, yet the row is visible with 300: transaction 805 is marked **aborted** in the commit log ( `pg_xact` ), so its `xmax` is ignored. PostgreSQL rolls back instantly by marking, not undoing.
5. `VACUUM` removed the dead versions: slots 3 and 5 (the aborted version) are unused ( `lp_flags = 0` ) and slot 1 became a **redirect** ( `lp_flags = 2` ) to slot 4 — a *HOT* (heap-only tuple) chain, so the index can keep pointing at slot 1.

**Two sessions** — the `mvcc` scenario of the driver shows what *another* transaction sees while an update is uncommitted:

```
=== MVCC: xmin / xmax seen from two sessions ===
  A| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1697, 0, 500.00)
  A| BEGIN  -> BEGIN
  A| UPDATE accounts SET balance = 450 WHERE id = 1  -> UPDATE 1
  A| SELECT txid_current()  -> 1698
  A| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1698, 0, 450.00)
  B| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1697, 1698, 500.00)
  A| COMMIT  -> COMMIT
  B| SELECT xmin, xmax, balance FROM accounts WHERE id = 1  -> (1698, 0, 450.00)
```

While A's transaction 1698 is open, **two versions** exist: A sees its new one (450); B sees the **old** one (500) with `xmax = 1698` – "being replaced by an uncommitted transaction, so still visible to me". After A commits, B's next statement sees 450. No one waited.

## Trade-offs

| Advantages | Costs |
|---|---|
| Readers and writers don't block each other | Extra storage for old versions; tables and indexes can **bloat** |
| Consistent snapshots for reports and backups ( `pg_dump` uses one) | Needs garbage collection (autovacuum must keep up) |
| Cheap rollback in PostgreSQL (mark aborted) | Long-running transactions keep old versions alive for everyone |
| Basis for snapshot isolation and SSI | Snapshot isolation alone still allows write skew (§13.8) |

**Two designs.** PostgreSQL keeps every version **in the table** (vacuum later). MySQL InnoDB and Oracle update **in place** and rebuild older versions from an **undo log** – less bloat, more work for long readers ("snapshot too old" in Oracle).

## Common mistakes

- Leaving a session "idle in transaction" for hours: vacuum can't remove versions newer than its snapshot, so tables bloat. Set `idle_in_transaction_session_timeout`.
- Disabling autovacuum: besides bloat, PostgreSQL's 32-bit transaction IDs must be "frozen" by vacuum before they wrap around (~2 billion transactions), or the server stops accepting writes.
- Thinking MVCC means "no locks at all" – writes to the same row still lock and wait.

## Real-world examples

Shoppers browse while prices change, customers view balances during batch posting, and `pg_dump` takes a coherent backup while the database keeps running – each of these works because readers get snapshots.

## In the interview

> **30-second answer:** "Keep multiple row versions tagged with the writing transaction; readers pick the version visible to their snapshot, so readers and writers don't block each other. In Postgres an update inserts a tuple and sets xmax on the old one; VACUUM removes dead tuples. Costs: bloat, garbage collection, and long transactions hurt."

- *Non-zero `xmax` on a visible row?* Its deleter/updater hasn't committed or aborted – or it's only a row lock.

## Self-check

1. What do `xmin` and `xmax` record? → The creating transaction and the deleting/replacing (or locking) transaction.
2. Why did the updated row get a new `ctid`? → An UPDATE writes a new tuple version elsewhere in the page.

3. What makes a rolled-back change invisible in PostgreSQL? → The transaction is marked aborted in the commit log; its versions are ignored.

---

## 13.15 Putting it together

### A practical checklist

1. Keep transactions **short** — never open across user think time or slow network calls.
2. Prefer **atomic statements** (`SET n = n - 1 WHERE n > 0`) over read-then-write in code; otherwise `FOR UPDATE`, or a **version column** for long gaps.
3. Declare every invariant you can as a **constraint** (unique, check, foreign key, exclusion); for cross-row rules use **Serializable + retry** or lock the rows the decision depends on.
4. **Retry** the whole transaction on SQLSTATE 40001 and 40P01, with backoff and a limit.
5. Lock rows in a **consistent order**; set `lock_timeout` for migrations and `idle_in_transaction_session_timeout` to protect vacuum.
6. For consistent multi-query reports use `REPEATABLE READ READ ONLY`.

Deeper theory: DDIA ch. 7 notes. Thread-level locking in application code: LLD handbook concurrency concepts and concurrency problems. Distributed transactions and replication: chapter 15; transactions from Python/Java: chapter 14. A one-page summary of isolation levels, anomalies and lock choices is in chapter 20, §20.12.

---

## Chapter TL;DR

- A **transaction** is all-or-nothing; write it with `BEGIN … COMMIT/ROLLBACK`, using savepoints for partial undo.
- **ACID:** atomicity (logs/shadow copies), consistency (constraints + code), isolation (locks/MVCC/SSI), durability (WAL + fsync + checkpoints + replication).
- **Anomalies:** dirty read, dirty write, non-repeatable read/read skew, phantom, lost update, write skew. PostgreSQL never allows dirty reads/writes; Read Committed allows the next four; Repeatable Read (snapshot isolation) stops all but write skew; Serializable (SSI) stops everything by aborting with 40001.
- **Lost updates:** prefer atomic SQL updates; otherwise `FOR UPDATE`, Repeatable Read + retry, or a version column (CAS) for long think times.
- **Concurrency control:** pessimistic (S/X/intention locks, 2PL, predicate/next-key locks) vs optimistic (validate at commit) vs timestamp ordering vs **MVCC** (PostgreSQL's `xmin`/`xmax`, cleaned by VACUUM).
- **Deadlocks** are detected (40P01) and prevented by consistent lock ordering; **retry** 40001/40P01.
- `FOR UPDATE SKIP LOCKED` turns a table into a safe multi-worker job queue.

Next: Chapter 14 — Advanced SQL and using SQL from code, which runs these transactions from Python and Java. For quick revision, see the transactions sheet in Chapter 20.